

# PKU–Zurich PhD Summer School on Machine Learning for Macroeconomics and Finance

Automatic differentiation for DEQNs: from the Lagrangian primitive to two-tape  
Euler residuals

Simon Scheidegger

HEC Lausanne · University of Zurich

July 6–10, 2026 · Beijing, China

Session 3 — a primer for the autodiff notebooks (03\_02\_Brock\_Mirman\_AutoDiff\_DEQN,  
03\_03\_Brock\_Mirman\_Uncertainty\_AutoDiff\_DEQN).

[https://liboecon.com/2026\\_summer\\_school.html](https://liboecon.com/2026_summer_school.html)

# Roadmap

1. **Motivation.** Where do derivatives appear in our models?
2. **Three ways to differentiate.** Symbolic, finite differences, autodiff.
3. **How autodiff works** – computational graph, forward vs reverse mode.
4. **Analytical warm-ups.** CRRA utility, Cobb–Douglas, capital adjustment cost.
5. **Brock–Mirman by hand.** Lagrangian, FOC, Bellman, envelope theorem.
6. **Brock–Mirman by autodiff.** The one-line Euler residual; notebooks 02, 03.
7. **Pitfalls of autodiff.** What it can (and cannot) do for you.
8. **Takeaways.**

## Prerequisites

A little calculus, a little Python. *No prior exposure to autodiff assumed.*

# Learning objectives

**By the end of this lecture you should be able to:**

- ▶ Implement automatic differentiation to compute exact first and second derivatives of arbitrary code for economic models
- ▶ Compare symbolic, finite-difference, and autodiff approaches and diagnose failure modes in each
- ▶ Derive first-order conditions and envelope-theorem expressions using autodiff instead of hand algebra
- ▶ Build a two-tape computation graph to compute Euler residuals directly from the Lagrangian
- ▶ Deploy gradient-descent optimization on neural-network policies using exact gradients computed via reverse-mode autodiff

# Derivatives are *everywhere* in macro-finance

## Where they show up

- ▶ **First-order conditions** of optimization problems (consumer, firm, planner).
- ▶ The **envelope theorem** relating  $V'(\cdot)$  to  $\partial \mathcal{L} / \partial (\text{state})$ .
- ▶ **Gradient descent** to train neural networks.
- ▶ **Sensitivity analysis / Greeks / UQ**: how does output  $y$  move with input  $x$ ?
- ▶ **Newton / quasi-Newton** solvers for equilibrium systems.
- ▶ **Implicit differentiation** for comparative statics.

## The pen-and-paper tax

Every time we write a new model we currently:

1. Write the Lagrangian.
2. Take FOCs by hand.
3. Apply the envelope theorem by hand.
4. Code the resulting expressions.

Change utility function  $\Rightarrow$  redo steps 1–4.

Change production function  $\Rightarrow$  redo steps 1–4.

One typo  $\Rightarrow$  wrong solver, silently.

**Goal of today:** write the model once, let the computer do calculus.

# Four reasons to care today

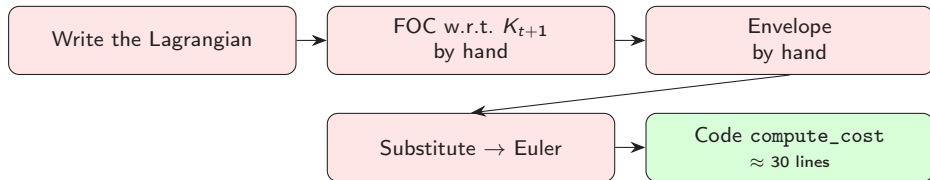
1. **FOCs of dynamic problems.** Euler equation asks for  $\partial u / \partial c_t$ ,  $\partial V / \partial K_{t+1}$ . Both are derivatives of things we already coded.
2. **Envelope theorem.**  $V'(K) = \partial \mathcal{L} / \partial K$  at the optimum, where  $\mathcal{L}$  is the Lagrangian. Another derivative of something we already coded.
3. **Neural-network training.**  $\partial \text{loss} / \partial \theta$  for *millions* of parameters  $\theta$ . Can't do this by hand.
4. **Sensitivities and uncertainty quantification.**  $\partial(\text{policy}) / \partial(\text{parameter})$  for robustness, comparative statics, and deep UQ (Day 8).

## Today's punchline

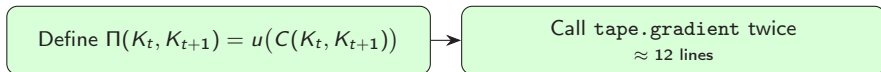
For *all* these uses, `tf.GradientTape` (or `jax.grad`, or `torch.autograd`) replaces pages of algebra. And it is *exact* – not approximate.

# Preview: the pen-and-paper tax, visualised

The original hand-derived Brock–Mirman notebook (lecture\_03\_01\_Brock\_Mirman\_1972\_DEQN; the autodiff version is lecture\_07\_02\_Brock\_Mirman\_AutoDiff\_DEQN) needed *two pages* of algebra before we could code the loss:



Notebook 02 (same model, autodiff) collapses all four red boxes into one function:



**Everything we learn today leads to that green box on the right.**

# Three ways to take a derivative

Given a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  and a point  $x_0$ , we can compute  $\nabla f(x_0)$  in three different ways:

|                       | Symbolic           | Finite differences         | Autodiff                    |
|-----------------------|--------------------|----------------------------|-----------------------------|
| <i>Tool</i>           | Mathematica, SymPy | a calculator, any language | PyTorch/TF/JAX              |
| <i>Accuracy</i>       | exact              | $O(h)$ or $O(h^2)$         | exact (machine $\epsilon$ ) |
| <i>Cost/dim</i>       | linear, blows up   | $n + 1$ fcn evals          | $\sim 2 \times$ one eval    |
| <i>Expr. swell</i>    | yes, often bad     | no                         | no                          |
| <i>Works on code?</i> | limited            | yes                        | yes                         |
| <i>Step size?</i>     | no                 | yes – hard!                | no                          |

- We briefly meet each; focus is on **finite differences** (the historical default) and **autodiff** (the modern default for ML and macro).

# Method 1: symbolic differentiation

**Idea:** manipulate an algebraic expression to produce a new algebraic expression.

## Small example

```
>>> from sympy import symbols, sin, log, diff
>>> x = symbols('x')
>>> f = sin(x**2) + log(1 + x**3)
>>> diff(f, x)
2*x*cos(x**2) + 3*x**2/(x**3 + 1)
```

### Strengths

- ▶ Exact, human-readable answer.
- ▶ Great for *one* clean formula.
- ▶ Can reveal analytical structure (cancellations, monotonicity).

### Weaknesses

- ▶ **Expression swell**: the derivative of a product of  $n$  terms has  $n$  summands, each more complex than the original.
- ▶ Cannot differentiate arbitrary *code* (loops, branches).



# Method 2: finite differences

**Idea:** approximate the derivative by the definition itself.

$$f'(x_0) \approx \underbrace{\frac{f(x_0 + h) - f(x_0)}{h}}_{\text{forward}} \quad \text{or} \quad \underbrace{\frac{f(x_0 + h) - f(x_0 - h)}{2h}}_{\text{central, accuracy } O(h^2)}$$

```
def fd_grad(f, x, h=1e-6):  
    return (f(x + h) - f(x - h)) / (2*h)
```

**Pros.** Trivial to implement; treats  $f$  as a black box.

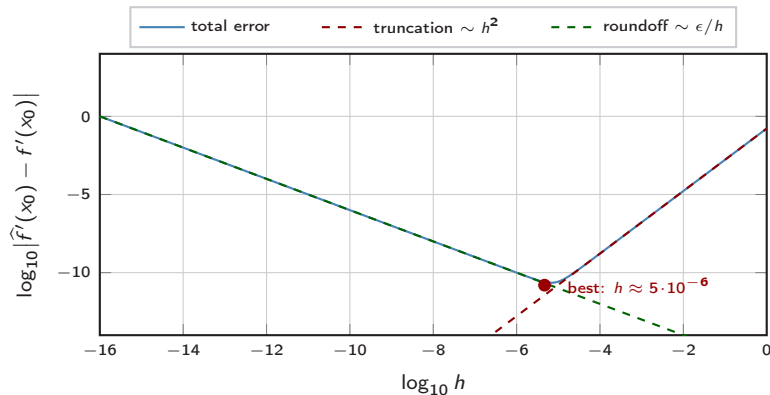
**Cons.** Two sources of error fight each other:

- ▶ **Truncation error.** Taylor:  $f(x + h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \dots$ ; the discarded terms give error  $O(h)$  (or  $O(h^2)$  for central).
- ▶ **Rounding error.** Computer stores numbers with  $\sim 16$  decimal digits. Computing  $f(x + h) - f(x - h)$  subtracts two nearly equal numbers  $\Rightarrow$  *catastrophic cancellation*; error roughly  $\epsilon/h$ .

Optimal  $h$  is a *compromise*, and it ruins your accuracy.

# Finite differences: the classic U-curve

For central FD applied to  $f(x) = \exp(x)$  at  $x = 1$ :

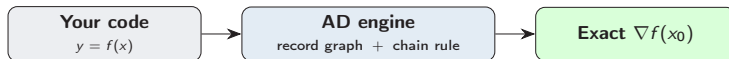


- ▶ **Best achievable error** (1st derivative, central FD):  $\sim 10^{-11}$ ; we lose 5–8 digits of precision.
- ▶ *Second derivatives*: lose  $\sim 10$  digits. Hessians are essentially numerical noise.

# Method 3: automatic differentiation – the elevator pitch

## Key insight

Every program is a composition of **elementary operations** ( $+$ ,  $-$ ,  $\times$ ,  $/$ ,  $\exp$ ,  $\log$ ,  $\sin$ , ...) whose derivatives we know in closed form. The chain rule mechanically combines these into the derivative of *any* program.



- ▶ **Exact to machine precision.** No truncation, no cancellation.
- ▶ **Works on code**, not just formulas. If it *runs*, it can be differentiated (with caveats we'll see).
- ▶ **No step size**, no tuning.
- ▶ **Reverse-mode cost**  $\approx 2\text{--}4\times$  the cost of evaluating  $f$ , *independent of dimension*.

# Autodiff in plain English

Three statements, in order. None of them is technical.

1. **Every program is a chain of elementary operations.** When you compute  $y = x^2 + \sin(x)$ , the computer doesn't see the formula. It sees: "square  $x$ , take sine of  $x$ , add the two". That's three elementary steps, each with a known derivative.
2. **The chain rule composes derivatives.** If  $y$  is built from  $x$  via three elementary steps, then  $dy/dx$  is built from those three local derivatives by the chain rule – mechanically. We learned this in calculus class; we now ask the computer to do it.
3. **Two passes, one cost.** The computer can apply the chain rule *at the same time* it evaluates  $f$  (**forward mode**) or *after* evaluating  $f$ , walking backwards (**reverse mode**). Either way, the answer is exact and the cost is a small multiple of evaluating  $f$  once.

## The whole idea

Autodiff is the chain rule, applied automatically to whatever sequence of elementary operations your code happens to execute. *There is no clever new mathematics.* Just bookkeeping.

# Where do the local derivatives come from? A tiny built-in table

**Punchline.** The computer doesn't "know calculus". It knows a short **hardcoded table** of derivatives for *elementary* operations. Every program is built from these; the chain rule composes the pieces.

| Primitive            | Rule (built into the framework)                            | Where written                          |
|----------------------|------------------------------------------------------------|----------------------------------------|
| $y = x + z$          | $\partial y / \partial x = 1, \partial y / \partial z = 1$ | <code>tf.math.add.gradient</code>      |
| $y = x \cdot z$      | $\partial y / \partial x = z, \partial y / \partial z = x$ | <code>tf.math.multiply.gradient</code> |
| $y = x^\alpha$       | $\partial y / \partial x = \alpha x^{\alpha-1}$            | <code>tf.math.pow.gradient</code>      |
| $y = \exp(x)$        | $\partial y / \partial x = \exp(x)$                        | <code>tf.math.exp.gradient</code>      |
| $y = \log(x)$        | $\partial y / \partial x = 1/x$                            | <code>tf.math.log.gradient</code>      |
| $y = \sin(x)$        | $\partial y / \partial x = \cos(x)$                        | <code>tf.math.sin.gradient</code>      |
| $y = \cos(x)$        | $\partial y / \partial x = -\sin(x)$                       | <code>tf.math.cos.gradient</code>      |
| $y = \text{ReLU}(x)$ | $\partial y / \partial x = 1\{x > 0\}$                     | <code>tf.nn.relu.gradient</code>       |

That's the whole trick. TensorFlow (and PyTorch, and JAX) ship with a few hundred of these rules. Your code composes them; the chain rule composes their derivatives.

## In one sentence

The computer "knows"  $d(\sin x)/dx = \cos x$  because *someone typed that rule into the framework*. We compose rules; we never re-derive them.

# Why the chain rule is enough: a 30-second worked example

Compute  $\frac{dy}{dx}$  for  $y = x^2 + \sin(x)$  at  $x = 2$ , *one elementary step at a time*.

**Decomposition.** Two intermediate variables, one per elementary operation on  $x$ :

$$v_1 := x^2, \quad v_2 := \sin(x), \quad y = v_1 + v_2.$$

| Step | Elementary operation                                   | Local derivative                                                 |
|------|--------------------------------------------------------|------------------------------------------------------------------|
| (1)  | $v_1 = x^2; \ x = 2 \Rightarrow v_1 = 4$               | $\partial v_1 / \partial x = 2x = 4$                             |
| (2)  | $v_2 = \sin(x); \ x = 2 \Rightarrow v_2 \approx 0.909$ | $\partial v_2 / \partial x = \cos(x) \approx -0.416$             |
| (3)  | $y = v_1 + v_2 \approx 4.909$                          | $\partial y / \partial v_1 = 1, \ \partial y / \partial v_2 = 1$ |

**Chain rule:**  $\frac{dy}{dx} = \frac{\partial y}{\partial v_1} \frac{\partial v_1}{\partial x} + \frac{\partial y}{\partial v_2} \frac{\partial v_2}{\partial x} = 1 \cdot 4 + 1 \cdot (-0.416) = 3.584$ .

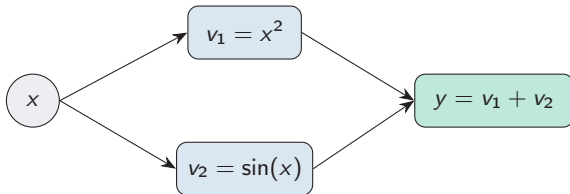
**Cross-check:**  $f'(x) = 2x + \cos(x)$ , so  $f'(2) = 4 + \cos(2) \approx 3.584$ . **Same answer.**

**Take-away.** No new mathematics, just bookkeeping along the chain. Next: picture it as a graph; then forward vs. reverse mode.

# Same example, drawn as a computational graph

The three elementary operations from the previous slide, as a directed graph:  
*nodes* = operations, *arrows* = intermediate values.

$$y = f(x) = x^2 + \sin(x) \quad \text{at} \quad x = 2.$$

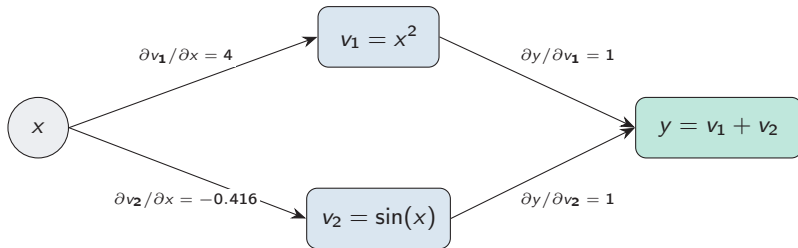


- ▶ **Nodes:** the same three elementary operations as in the table.
- ▶ **Arrows:** intermediate values  $v_1, v_2$  flowing between operations.
- ▶ **Local derivatives live on the arrows:**  $\partial v_1 / \partial x = 2x$ ,  $\partial v_2 / \partial x = \cos(x)$ ,  $\partial y / \partial v_i = 1$ .

The graph is a *visual organiser* for the chain rule – and lets us state *forward* and *reverse* mode cleanly on the next slides.

# The graph carries the local derivatives – both modes use them

Same graph, annotated with each edge's local derivative:



Two strategies for combining the edges into  $dy/dx$ .

- **Forward mode:** walk *left to right* (with  $f$ ); accumulate  $\dot{v} = \partial v / \partial x$  from parent  $\dot{u}$ . Read  $\dot{y}$ .
- **Reverse mode:** walk *right to left* (after  $f$ ); accumulate  $\bar{v} = \partial y / \partial v$  from child  $\bar{w}$ . Read  $\bar{x}$ .

Both give the same number; they differ in *when* bookkeeping happens and *which dimension* they scale with – detailed next.



# Forward mode: propagate derivatives *with* the computation

**Big idea.** Carry a **derivative tag**  $\dot{v}$  next to every value  $v$ , and update both in the same forward pass.

The chain rule, applied locally to each elementary operation

$$v = g(u) \Rightarrow \dot{v} = g'(u) \dot{u}, \quad v = g(u_1, u_2) \Rightarrow \dot{v} = g_1 \dot{u}_1 + g_2 \dot{u}_2.$$

**Recipe.**

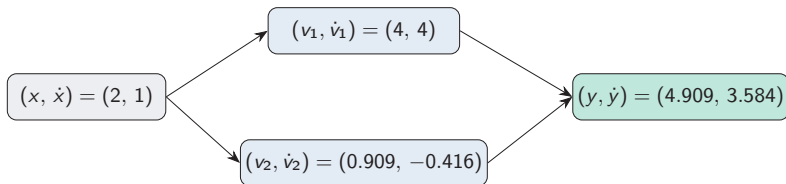
1. Replace each  $v$  in the program by the pair  $(v, \dot{v})$  (a **dual number**).
2. Seed at the input:  $\dot{x} = 1$ .
3. Run forward: each operation updates both the value and its tag.
4. Read  $\dot{y}$  at the output – this is  $dy/dx$ , exact.

**Why “forward”?** The derivative moves *with* the program. **Why correct?** The chain rule, applied at every node. Next slide: in action on  $y = x^2 + \sin(x)$ .

# Forward mode: worked example on $y = x^2 + \sin(x)$ at $x = 2$

**Seed** at the input:  $(x, \dot{x}) = (2, 1)$ . Each elementary operation updates both fields via the chain rule:

- ▶  $v_1 = x^2 \Rightarrow \dot{v}_1 = 2x \dot{x} = 4$ . So  $(v_1, \dot{v}_1) = (4, 4)$ .
- ▶  $v_2 = \sin(x) \Rightarrow \dot{v}_2 = \cos(x) \dot{x} \approx -0.416$ . So  $(v_2, \dot{v}_2) = (0.909, -0.416)$ .
- ▶  $y = v_1 + v_2 \Rightarrow \dot{y} = \dot{v}_1 + \dot{v}_2 = 3.584$ .

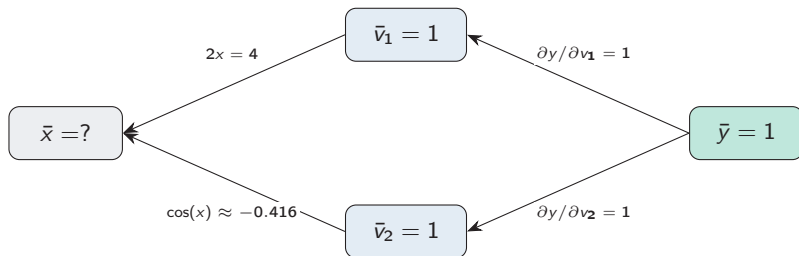


**Read out**  $\dot{y} = dy/dx = 3.584$ . Cross-check:  $f'(2) = 4 + \cos(2) = 3.584$ . **Exact.**

- ▶ **Cost:**  $\sim 2\times$  one call to  $f$ .
- ▶ **Scales with #inputs:** for  $\nabla f$  with  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , need  $n$  forward passes. Bad for neural nets; *reverse mode* (next) fixes this.

# Reverse mode (= backprop): propagate *backwards*

**Idea.** First evaluate  $f$  forward and *remember* the graph. Then walk backwards, carrying the sensitivity  $\bar{v} = \partial y / \partial v$  along each edge. Seed  $\bar{y} = 1$  at the output.



**Rule.**  $\bar{x} = \sum_{i: x \rightarrow v_i} \bar{v}_i \cdot \frac{\partial v_i}{\partial x}$ . So

$$\bar{x} = \bar{v}_1 \cdot 4 + \bar{v}_2 \cdot (-0.416) = 1 \cdot 4 + 1 \cdot (-0.416) = 3.584.$$

- **Cost:**  $\sim 2-4 T_f$ ; memory  $\propto$  graph size (one issue we'll revisit).
- **Scales with #outputs:** one reverse pass per output – great for scalar losses with many inputs.
- **This is what TF/PyTorch/JAX do under the hood for neural nets.**

# Which mode to use?

For  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ;  $T_f$  = cost of one forward evaluation of  $f$ .

| Setting                                  | Dimensions            | Mode of choice            | Cost (full Jac.) |
|------------------------------------------|-----------------------|---------------------------|------------------|
| Neural-net training                      | $n \sim 10^6$ , $m=1$ | <b>Reverse</b> (backprop) | $2-4 T_f$        |
| Sensitivity, many inputs<br>→ one output | $n$ large, $m=1$      | Reverse                   | $2-4 T_f$        |
| Tangents, ODE solvers                    | $n=1$ , $m$ large     | <b>Forward</b>            | $\sim 2 T_f$     |
| Square Jacobian                          | $n = m$               | Either works              | $\sim n T_f$     |

## For DEQNs (and most of this course)

Scalar loss,  $n \sim 10^3$  to  $10^5$  parameters. **Reverse mode wins**: one pass at  $\sim 4 T_f$  gives the gradient w.r.t. *all*  $n$  parameters. Frameworks (`tf.GradientTape`, `torch.autograd`, `jax.grad`) default to reverse mode.

# Warm-up 1: CRRA marginal utility

Isoelastic (CRRA) utility with  $\gamma = 2$ :

$$u(c) = \frac{c^{1-\gamma}}{1-\gamma} = -\frac{1}{c}, \quad u'(c) = c^{-\gamma} = \frac{1}{c^2}, \quad u''(c) = -\gamma c^{-\gamma-1}.$$

```
import tensorflow as tf
gamma = 2.0
def u(c):                                # one-liner, model primitive
    return c**(1.0 - gamma) / (1.0 - gamma)

c = tf.Variable(3.0)
with tf.GradientTape(persistent=True) as tape:
    y = u(c)
    up = tape.gradient(y, c)             # first derivative via AD
    upp = tape.gradient(up, c)           # second derivative via AD
# hand: u'(3) = 1/9 = 0.1111...   u''(3) = -2/27 = -0.07407...
```

## Result

AD returns  $u'(3) \approx 0.11111111$  and  $u''(3) \approx -0.07407407$  to machine precision. **No hand derivation.**  
Change  $\gamma$  to 4: the code is unchanged.

# Warm-up 2: Cobb–Douglas marginal products

Production function with  $\alpha = 0.36$ :

$$f(K, L) = K^\alpha L^{1-\alpha}, \quad \partial_K f = \alpha K^{\alpha-1} L^{1-\alpha}, \quad \partial_L f = (1 - \alpha) K^\alpha L^{-\alpha}.$$

```
alpha = 0.36
def f(K, L):                                # model primitive
    return K**alpha * L**(1.0 - alpha)

K, L = tf.Variable(5.0), tf.Variable(2.0)
with tf.GradientTape() as tape:
    y = f(K, L)
grads = tape.gradient(y, [K, L])  # returns a list with both partials

# hand: f_K(5,2) = 0.36 * 5^(-0.64) * 2^(0.64) ~ 0.1832
#        f_L(5,2) = 0.64 * 5^(0.36) * 2^(-0.36) ~ 1.0461
```

## Take-away

One *tape*, one call, *both* partial derivatives. Replace Cobb–Douglas by CES: change the `def f(...)` line; the *gradient* line is unchanged.

## Warm-up 3: capital adjustment cost (by hand is annoying)

Convex adjustment cost, from Day 3's IRBC model:

$$\Gamma(K, K') = \frac{\kappa}{2} K \left( \frac{K'}{K} - 1 \right)^2.$$

By hand, **two partial derivatives**, each needing the *quotient* and *chain* rules:

$$\begin{aligned}\frac{\partial \Gamma}{\partial K'} &= \kappa K \left( \frac{K'}{K} - 1 \right) \cdot \frac{1}{K} = \kappa \left( \frac{K'}{K} - 1 \right), \\ \frac{\partial \Gamma}{\partial K} &= \frac{\kappa}{2} \left( \frac{K'}{K} - 1 \right)^2 + \kappa K \left( \frac{K'}{K} - 1 \right) \cdot \left( -\frac{K'}{K^2} \right) \\ &= \frac{\kappa}{2} \left( \frac{K'}{K} - 1 \right)^2 - \kappa \frac{K'}{K} \left( \frac{K'}{K} - 1 \right).\end{aligned}$$

This is the kind of algebra where *typos* live.

## Warm-up 3 (cont.): by autodiff

```
kappa = 0.3
def Gamma(K, Kp):                # model primitive -- ONE line
    return 0.5 * kappa * K * (Kp / K - 1.0)**2

K = tf.Variable(5.0)
Kp = tf.Variable(5.6)
with tf.GradientTape() as tape:
    g = Gamma(K, Kp)
dK, dKp = tape.gradient(g, [K, Kp])
# AD:  dK    ~  -0.0252  (matches hand)
#      dKp   ~   0.036   (matches hand)
```

### Observation

**Hand:** two paragraphs of algebra, one chance to typo a sign.

**AD:** one Python definition, zero chances to typo. Same answer to machine precision.

This is the small version of what happens on the full Brock–Mirman Lagrangian next.



# What we've learned about AD so far

1. **You write only the model primitive** – the utility function, the production function, the cost function, the Lagrangian. *Not* any derivative.
2. **AD returns the derivative to machine precision**, not an approximation.
3. **Adding a new partial derivative is free** (one more entry in the argument list of `tape.gradient`).
4. **Changing the functional form leaves the gradient code untouched.**
5. **Reverse mode is cheap:**  $\sim 2\text{--}4\times$  the forward pass, independent of the number of inputs.

## Coming up

Now we will apply this *exactly the same pattern* to the Brock–Mirman planner's problem. First: derive the Euler equation by hand, to see what we are replacing.

# The deterministic Brock–Mirman model

The social planner solves

$$\max_{\{C_t, K_{t+1}\}_{t \geq 0}} \sum_{t=0}^{\infty} \beta^t \ln(C_t) \quad \text{s.t.} \quad K_{t+1} + C_t = K_t^\alpha + (1 - \delta) K_t, \quad K_0 \text{ given.}$$

## Primitives

- Utility:  $u(C) = \ln C$ .
- Production:  $Y_t = K_t^\alpha$  (Cobb–Douglas,  $\alpha \in (0, 1)$ ).
- Depreciation rate:  $\delta \in [0, 1]$ .
- Discount factor:  $\beta \in (0, 1)$ .

## What we want

A *policy function*

$$K_{t+1} = g(K_t)$$

that maximises lifetime utility. With  $\delta = 1$  the answer is known:

$$g(K) = \alpha\beta K^\alpha.$$

We will *derive* the Euler equation that  $g$  must satisfy.

# The sequence-form Lagrangian

Attach a multiplier  $\beta^t \lambda_t$  to the resource constraint at every date:

Lagrangian (written out in full)

$$\mathcal{L} = \sum_{t=0}^{\infty} \beta^t \left[ \underbrace{\ln C_t}_{\text{utility}} + \lambda_t \underbrace{\left( K_t^\alpha + (1 - \delta) K_t \right)}_{\text{resources}} - C_t - K_{t+1} \right].$$

*Choice variables* at each date:  $C_t$  and  $K_{t+1}$ .

*State* at date  $t$ :  $K_t$ , coming in from the previous period.

**Plan.** Take FOCs with respect to each choice variable, combine with the constraint, and get a single difference equation in  $K_t$ .

# FOC with respect to $C_t$

Differentiate  $\mathcal{L}$  with respect to  $C_t$  (which appears only in the  $t$ -th summand):

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial C_t} &= \beta^t \left[ \frac{1}{C_t} - \lambda_t \right] = 0 \\ \implies \lambda_t &= \frac{1}{C_t} = u'(C_t).\end{aligned}$$

## Interpretation

The multiplier  $\lambda_t$  equals the *marginal utility of consumption*. Unsurprising – the planner is willing to pay  $u'(C_t)$  additional units of resources to get one more unit of consumption today.

# FOC with respect to $K_{t+1}$

$K_{t+1}$  appears *twice* in the Lagrangian: as the choice at date  $t$  (with coefficient  $-\beta^t \lambda_t$ ) and as the state at date  $t+1$  (inside  $K_{t+1}^\alpha + (1-\delta)K_{t+1}$ , with coefficient  $+\beta^{t+1} \lambda_{t+1}$ ).

$$\frac{\partial \mathcal{L}}{\partial K_{t+1}} = -\beta^t \lambda_t + \beta^{t+1} \lambda_{t+1} [\alpha K_{t+1}^{\alpha-1} + (1-\delta)] = 0$$
$$\implies \lambda_t = \beta \lambda_{t+1} \left[ \underbrace{\alpha K_{t+1}^{\alpha-1}}_{\text{MPK}_{t+1}} + 1 - \delta \right].$$

## Interpretation

One unit *not* consumed today (costing  $\lambda_t$  units of utility) becomes capital tomorrow, earning the gross rental  $\text{MPK}_{t+1} + 1 - \delta$  in resource units, which is then valued at tomorrow's multiplier  $\beta \lambda_{t+1}$ .

# Combine: the Euler equation

Substitute  $\lambda_t = u'(C_t) = 1/C_t$  from the first FOC into the second:

Euler equation (deterministic)

$$\frac{1}{C_t} = \beta \frac{1}{C_{t+1}} [\alpha K_{t+1}^{\alpha-1} + 1 - \delta].$$

Equivalently, writing  $r_{t+1} = \alpha K_{t+1}^{\alpha-1}$ ,

$$u'(C_t) = \beta u'(C_{t+1}) (1 - \delta + r_{t+1}).$$

- ▶ This is the *consumption smoothing* condition. Marginal utility today = discounted, rate-adjusted marginal utility tomorrow.
- ▶ This is the equation that `compute_cost` in the hand-derived Brock–Mirman notebook (`lecture_03_01`) minimises the squared residual of.

# Recursive (Bellman) form and the envelope theorem

The same problem has a recursive representation:

$$V(K) = \max_{K'} \left\{ \underbrace{u(K^\alpha + (1 - \delta)K - K')}_{=C(K, K')} + \beta V(K') \right\}.$$

Let  $g(K)$  be the optimal  $K'$ .

**Envelope theorem.** Differentiate  $V$ , use the FOC  $-u'(C) + \beta V'(g(K)) = 0$  to cancel terms:

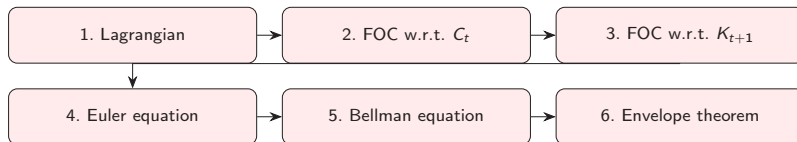
$$\begin{aligned} V'(K) &= u'(C) \cdot \underbrace{[\alpha K^{\alpha-1} + 1 - \delta]}_{\partial C / \partial K} + [-u'(C) + \beta V'(g(K))] \cdot g'(K) \\ &= u'(C) [\alpha K^{\alpha-1} + 1 - \delta]. \quad (\text{FOC kills the second bracket.}) \end{aligned}$$

## Envelope formula

$$V'(K_t) = u'(C_t) [\alpha K_t^{\alpha-1} + 1 - \delta].$$

*The derivative of  $V$  equals the partial of the Lagrangian w.r.t. the state – about to be exploited.*

# Recap: what we derived by hand



Each red box is *real work* – pen-and-paper algebra, every time we change  $u$  or  $Y$  or  $\delta$ .

## The autodiff promise

Define the **period payoff**

$$\Pi(K_t, K_{t+1}) := u(\underbrace{Y(K_t) + (1 - \delta)K_t - K_{t+1}}_{= C_t}).$$

Then **every red box is the derivative of the same function  $\Pi$** . Give  $\Pi$  to TensorFlow, and `tape.gradient` runs boxes 2, 3, and 6 for us; substitution (box 4) is done by how we *write the loss*. We will never re-do the algebra again. **Detailed treatment of  $\Pi$  on the next slide.**



# The single primitive $\Pi$

Period payoff – the only thing the user writes

$$\Pi(K_t, K_{t+1}) = u(\underbrace{K_t^\alpha + (1 - \delta) K_t - K_{t+1}}_{=C_t}).$$

**Slot notation:**  $\partial_i \Pi$  = partial derivative w.r.t. the  $i$ -th input slot; for  $\Pi(x, y)$ :  
 $\partial_1 \Pi(x, y) \equiv \frac{\partial \Pi}{\partial x}(x, y)$ ,  $\partial_2 \Pi(x, y) \equiv \frac{\partial \Pi}{\partial y}(x, y)$ .

Two facts from the previous slides:

- **FOC w.r.t.  $K_{t+1}$ :**  $\partial_2 \Pi(K_t, K_{t+1}) + \beta V'(K_{t+1}) = 0$  (slot 2 = today's choice).
- **Envelope:**  $V'(K) = \partial_1 \Pi(K, g(K))$  (slot 1 = the state).

Substituting the envelope (at  $K_{t+1}$ ) into the FOC:

Autodiff Euler residual (deterministic)

$$\underbrace{\partial_2 \Pi(K_t, K_{t+1})}_{\text{FOC term}} + \beta \underbrace{\partial_1 \Pi(K_{t+1}, K_{t+2})}_{\text{envelope term}} = 0.$$

Both terms are gradients of the same  $\Pi$ . TensorFlow computes them.

# Side-by-side: hand-derived vs autodiff loss

## Hand-derived (lecture\_03\_01)

```
def compute_cost(X, nn):
    K_t = X
    Y_t = K_t**alpha
    s_t = nn(K_t)
    K_tp1 = (1-delta)*K_t + Y_t*s_t
    C_t = Y_t - Y_t*s_t

    Y_tp1 = K_tp1**alpha
    s_tp1 = nn(K_tp1)
    C_tp1 = Y_tp1 - s_tp1*Y_tp1
    R_tp1 = alpha*K_tp1**(alpha-1)

    res = -1/C_t + beta/C_tp1 \
          * (1 - delta + R_tp1)
    return tf.reduce_mean(res**2)
```

Encodes:  $u'(C_t)$ ,  $u'(C_{t+1})$ , MPK, envelope – all done by the author on paper first.

## Autodiff (lecture\_07\_02)

```
def Pi(K_in, K_out):
    Y = K_in**alpha
    C = Y + (1-delta)*K_in - K_out
    return tf.math.log(C)

def compute_cost(X, nn):
    K_t = X
    K_tp1 = (1-delta)*K_t + (K_t**alpha)*nn(K_t)
    K_tp2 = (1-delta)*K_tp1 + (K_tp1**alpha)*nn(K_tp1)
    with tf.GradientTape() as t1:
        t1.watch(K_tp1)
        pi_t = Pi(K_t, K_tp1)
        foc = t1.gradient(pi_t, K_tp1)
    with tf.GradientTape() as t2:
        t2.watch(K_tp1)
        pi_tp1 = Pi(K_tp1, K_tp2)
        env = t2.gradient(pi_tp1, K_tp1)
    return tf.reduce_mean((foc + beta*env)**2)
```

Encodes: the primitive  $\Pi$ . That's it.

Change  $u$ ,  $Y$ , or  $\delta$ ? Only the  $\Pi$  function changes. Right column is unchanged.

# Adding uncertainty: the stochastic case

The stochastic Brock–Mirman model (autodiff version: notebook 03) adds an AR(1) shock  $z_t$ :

$$Y_t = z_t K_t^\alpha, \quad \log z_{t+1} = \rho \log z_t + \sigma \varepsilon_{t+1}, \quad \varepsilon_{t+1} \sim \mathcal{N}(0, 1).$$

The Euler equation gets an expectation on the right:

$$u'(C_t) = \beta \mathbb{E}_t[u'(C_{t+1})(1 - \delta + r_{t+1})].$$

**Autodiff template.** Replace  $\partial_1 \Pi$  by its conditional expectation, approximated by Gauss–Hermite:

$$\partial_2 \Pi(K_t, K_{t+1}, z_t) + \beta \sum_{q=1}^Q w_q \partial_1 \Pi(K_{t+1}, K_{t+2}, z_{t+1}^{(q)}) = 0.$$

## Code change

In `Pi(K_in, K_out, z_in)` multiply  $Y$  by  $z\_in$ . In the loss, **sum the envelope term over the  $Q$  quadrature nodes**. That's the entire extension.

# Cross-checks from the notebooks

From notebooks 03\_02\_Brock\_Mirman\_AutoDiff\_DEQN and  
03\_03\_Brock\_Mirman\_Uncertainty\_AutoDiff\_DEQN:

| Cross-check                                                            | Result                                    | Verdict           |
|------------------------------------------------------------------------|-------------------------------------------|-------------------|
| Autodiff vs hand residual (deterministic)                              | $\max  \Delta  = 4.8 \times 10^{-7}$      | machine precision |
| Autodiff vs hand residual (stochastic, GH)                             | $\max  \Delta  = 8.1 \times 10^{-6}$      | machine precision |
| Trained policy vs analytical $\alpha\beta K^\alpha$ (det.)             | $\max \text{rel err } 9.6 \times 10^{-5}$ | policy recovered  |
| Trained policy vs $\alpha\beta z K^\alpha$ ( $\delta=1$ , $25^2$ grid) | $\max \text{rel err } 1.7 \times 10^{-3}$ | policy recovered  |

## What this says

The autodiff loss is not an *approximation* of the textbook Euler equation – it *is* the textbook Euler equation, computed by a different mechanism. Training works as well as the hand-derived Brock–Mirman notebooks. We simply stopped writing algebra.

# When the template really pays off

Pattern:  $\partial_2 \Pi + \beta \mathbb{E}[\partial_1 \Pi] = 0$  scales *without algebra*.

- ▶ **Multiple states.**  $\Pi(K_t, z_t, K_{t+1})$ . Same code: `tape.gradient(pi, K_out)`.
- ▶ **Multiple controls.** Let  $a = (K', L)$ . AD returns gradient w.r.t. each in one call.
- ▶ **Heterogeneous agents.**  $\Pi$  becomes a sum over  $i$ ; gradient of a sum is a sum of gradients.
- ▶ **OLG.** A separate  $\Pi$  per age cohort; gradients stack.
- ▶ **Irreversibility constraints** (IRBC). Fischer–Burmeister residuals sit alongside; the core Euler part still autodiffs.

Think of it this way

You are now equipped to write **the loss of any DEQN-style model** by writing down its period payoff. The rest of the course is about *which*  $\Pi$  goes with which economic question.

# Further example (self-study): IRBC under autodiff

## Notebook 03\_04\_IRBC\_AutoDiff\_DEQN.ipynb

The Day 3 baseline IRBC model (multi-country, adjustment costs, irreversibility) re-solved using the same  $\partial_2 \Pi + \beta \mathbb{E}[\partial_1 \Pi]$  template, scaled from one Brock–Mirman state to a multi-equation planner Lagrangian.

- ▶ **One primitive per country.**  $\Pi^j(k_{\text{in}}, k'_{\text{out}}, z; \lambda, \mu^j)$  – the planner Lagrangian. Hand-derived helpers `production_k`, `mpk`, `adj_cost_k`, `adj_cost_kp` disappear: gone, replaced by `tape.gradient`.
- ▶ **What stays hand-coded.** The aggregate resource constraint (algebra) and Fischer–Burmeister (KKT complementarity for irreversibility) – autodiff has nothing to add.
- ▶ **Cross-check 1.** Autodiff Euler residual versus the Day 3 hand-derived residual on the same untrained network:  $\leq 1.2 \times 10^{-7}$  (float32 epsilon).
- ▶ **Side-by-side experiment.** Exogenous (Approach A) vs simulation-based (Approach B) sampling under the autodiff loss, identical architecture and budget, no ReLoBRaLo.

Provided as a further example / self-study; not covered in this in-class slot.

# Pitfalls – a preview

AD is exact and cheap, but not a silver bullet. Five pitfalls, illustrated in order:

1. **Non-differentiable kinks.** ReLU at 0,  $|x|$ , max, indicators.
2. **Singular gradients at the boundary.**  $\log x$  at  $x = 0$ ,  $1/x$ ,  $\sqrt{x}$ .
3. **Reverse-mode memory blow-up.** Long unrolled simulations.
4. **No structural insight.** AD returns numbers, not formulas.
5. `stop_gradient` **subtleties.** Envelope-theorem substitutions that only hold *at the optimum*.

None of these are catastrophic – but being aware of them is what separates a student who *uses* AD from one who *debugs* AD.

# Pitfall 1: non-differentiable kinks

ReLU is  $\max(0, x)$ ; its derivative is 0 for  $x < 0$ , 1 for  $x > 0$ , *undefined* at  $x = 0$ .

Frameworks return a *subgradient* by convention:

```
x = tf.Variable(0.0)
with tf.GradientTape() as tape:
    y = tf.nn.relu(x)
print(tape.gradient(y, x))    # tf: 0.0 ; torch: 0.0 as well
```

**Consequence.** If your loss lands *exactly* on a kink, SGD receives a zero gradient and *stalls*.

## Common offenders

- ▶ `relu`, `abs`, `maximum`, `minimum`
- ▶ `tf.where(cond, a, b)` – OK, but the *condition* is non-differentiable.
- ▶ Indicator functions  $1\{x > 0\}$ .
- ▶ `sort`, `argmax`, `round`.

## Fixes

- ▶ **Smooth the kink:** `softplus` for ReLU,  $\sqrt{x^2 + \epsilon^2}$  for  $|x|$ , `log-sum-exp` for `max`.
- ▶ **Fischer–Burmeister** for KKT complementarity (Day 3 IRBC).
- ▶ Add **slight noise** to inputs during training.



# Pitfall 2: singular gradients at the boundary

If the model pushes  $C \rightarrow 0$  during training,  $u(C) = \ln C \rightarrow -\infty$  and  $u'(C) = 1/C \rightarrow +\infty$ : AD returns a huge number, SGD takes a huge step, the network explodes.

```
C = tf.Variable(1e-8)                                # nearly zero consumption
with tf.GradientTape() as tape:
    y = tf.math.log(C)
print(tape.gradient(y, C))                            # ~ 1e8 --> NaN after a few steps
```

Cures that keep the math honest:

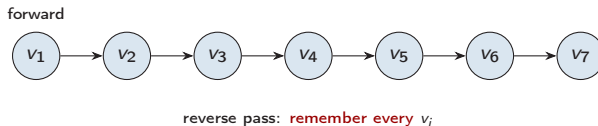
- ▶ **Reparameterise** so  $C > 0$  by architecture. The Brock–Mirman DEQN notebooks use a sigmoid on the savings rate  $s \in (0, 1)$ , guaranteeing  $C > 0$  and  $K' > 0$ .
- ▶ **Stable primitives**: `tf.math.log1p`, `tf.math.softplus`, `tf.math.xlogy`.
- ▶ `tf.clip_by_value` is a last resort, not a design choice.

## Moral

Hand-derivation often reveals a cancellation AD cannot see. *You* still need to design primitives that are numerically stable.

# Pitfall 3: reverse-mode memory

Reverse mode stores the *entire forward graph* so it can traverse it backwards.



**When this bites.**

- ▶ 10 000-step trajectory with back-prop through all of it:  $\text{memory} \propto T \times (\text{state dim})$ ; OOM on a laptop.
- ▶ PINN training with deep nets and many collocation points: tens of millions of graph nodes.

**Mitigations.**

- ▶ **Path averaging / mini-batching** over trajectory segments (the simulation-based Brock–Mirman notebook).
- ▶ **Gradient checkpointing**: recompute parts of the graph on the way back (`tf.recompute_grad`).
- ▶ **Truncated back-prop-through-time** for long recurrences.
- ▶ **Forward mode** when `#inputs` is small.

# Pitfall 4: no structural insight

AD returns a *number*, not an *expression*. Whenever the expression is worth seeing, AD hides it.

## Examples of useful structure

- ▶ **Cancellations:**  $u'(C_t) - \beta u'(C_{t+1})r$  has the same sign structure as the gross growth rate – obvious from the formula, invisible in a number.
- ▶ **Monotonicity:**  $V'(K) > 0$  is an inequality you can often read off  $\partial \mathcal{L} / \partial K$ .
- ▶ **Scaling laws:**  $\partial V / \partial A$  for TFP  $A$  is linear in  $A$  – useful for normalization.
- ▶ **Sufficient statistics:** derivative is only a function of certain summaries.

## Practical advice

- ▶ Do the hand derivation *once* on a toy version, so you *know* what the gradient should look like.
- ▶ Cross-check on a simple case: the autodiff number should agree with the closed-form formula to machine precision (this is exactly what the autodiff Brock–Mirman notebooks 02 and 03 do).
- ▶ Use AD to scale, not to replace, economic understanding.

# Pitfall 5: stop\_gradient and the envelope theorem

The envelope formula  $V'(K) = \partial_1 \Pi(K, g(K))$  is only valid *at the optimum*. What if we are *not yet* at the optimum, i.e. during training? **Two natural ways to code it, with different results off the optimum:**

1. **Treat  $K' = g(K)$  as a fixed function of  $K$**  (i.e. differentiate through the policy). This gives the *total* derivative, not the envelope; only correct when FOC is satisfied.
2. **Treat  $K'$  as an `tf.stop_gradient` input** (i.e. do *not* differentiate through the policy). This gives the *envelope* value directly – the formula we derived.

**At the optimum both coincide.** During training they differ; the second one is the one the hand derivation gives.

## What the notebooks do

In the autodiff Brock–Mirman notebooks 02 and 03 we *compute gradients of  $\Pi$  at values  $K_{t+1}, K_{t+2}$  that are produced by the policy*. We do *not* then additionally differentiate with respect to how the policy depends on  $K_t$ . Check: open the notebook and notice there is only one GradientTape per term, watching the *variable* we differentiate with respect to. That is the `stop_gradient` behavior, built in.

# Pitfalls: quick reference

| Pitfall                | Symptom                               | Remedy                                                              |
|------------------------|---------------------------------------|---------------------------------------------------------------------|
| Kinks (ReLU, abs, max) | gradient = 0 or undefined at the kink | smooth (softplus, FB), add noise                                    |
| Boundary singularities | NaNs, exploding gradients             | reparameterise domain; stable primitives                            |
| Reverse-mode memory    | OOM on long graphs                    | checkpointing, truncated BPTT, forward mode                         |
| Loss of insight        | hard to debug unexpected gradients    | hand derivation on a toy; cross-checks                              |
| Envelope vs total grad | wrong loss off the optimum            | differentiate w.r.t. the right variable; <code>stop_gradient</code> |

Autodiff replaces calculus. It does *not* replace thinking about the model.

# One slide to remember

The autodiff pattern for any recursive model

$$\Pi(\text{state, choice}) \text{ written once} \implies \partial_2 \Pi + \beta \mathbb{E}[\partial_1 \Pi] = 0.$$

- ▶ **Symbolic / FD / AD** all compute derivatives; only AD is exact, dimension-free, and works on code.
- ▶ **Forward vs reverse mode** is a choice of traversal direction on the computational graph. Reverse is the right default for DEQNs and neural networks.
- ▶ **The Lagrangian, the FOC, and the envelope theorem** of Brock–Mirman are all gradients of a single period payoff  $\Pi$ . Writing  $\Pi$  once removes all of them from the coding path.
- ▶ **Pitfalls** (kinks, singularities, memory, stop-gradient) are real but small if you know they exist.

From now on, every model in this course ships with an autodiff loss.

# Where to go next

## Run the notebooks

- ▶ 03\_01\_AutoDiff\_Analytical\_Examples: warm-ups + 2-D gradient fields from §4.
- ▶ 03\_02\_Brock\_Mirman\_AutoDiff\_DEQN: deterministic Brock–Mirman.
- ▶ 03\_03\_Brock\_Mirman\_Uncertainty\_AutoDiff\_DEQN: stochastic Brock–Mirman + Gauss–Hermite.
- ▶ 03\_04\_IRBC\_AutoDiff\_DEQN (*further example, self-study*): IRBC under the autodiff template – multi-country planner Lagrangian, machine-precision cross-check vs the IRBC session (Session 4).

All include cross-checks vs hand-derived loss and vs analytical solutions.

## Further reading

- ▶ Baydin, Pearlmutter, Radul & Siskind (2018), “Automatic differentiation in machine learning: a survey”, *JMLR* 18(153). Canonical overview.
- ▶ Griewank & Walther (2008), *Evaluating Derivatives* (SIAM). Textbook.
- ▶ JAX autodiff cookbook: [https://jax.readthedocs.io/en/latest/notebooks/autodiff\\_cookbook.html](https://jax.readthedocs.io/en/latest/notebooks/autodiff_cookbook.html).

Questions?